1. What is Git, and why is it used instead of other version control tools?

A: Git is a distributed version control system (DVCS) that tracks every change to code over time. It is widely used because every developer gets a full copy of the entire project history locally, making it faster, more reliable (no single point of failure), and allowing developers to work completely offline.

2. What is a Git repository, and what does it contain?

A: A Git repository is the central storage location for your project. It contains all the committed versions (snapshots) of your files, all branches, and the complete history of who changed what and when. The repository is stored locally in the hidden .git folder within your project directory.

3. How do you clone a repository from GitHub or any remote server?

A: You clone a repository using the command git clone <repository_url>. This downloads a complete copy of the remote repository and its entire history to your local machine, creating a new local project folder.

4. What is a Git commit, and what role does it play in version control?

A: A Git commit is a permanent snapshot of your project's files at a specific point in time. It plays the role of creating a reliable historical record. Each commit stores a unique ID (hash) and a descriptive message about the changes, building the chronological history of the project.

5. What is a Git branch, and why do developers use branches?

A: A Git branch is a lightweight pointer that identifies a line of development. Developers use branches to create independent workspaces for new features or bug fixes. This allows them to experiment and make changes without affecting the main, stable code until the changes are fully ready.

6. What is a Git remote, and how does it help in collaboration?

A: A Git remote is a version of the repository hosted on a network server (like GitHub or GitLab). It helps in collaboration by acting as a shared synchronization hub. Developers publish their local changes to the remote (git push) and download changes made by others (git pull).

7. What is the difference between git pull, git fetch, and git clone?

A: git clone is used only once to download the entire repository. git fetch downloads the latest changes from the remote but keeps them separate from your working files. git pull is a shortcut that performs a fetch and immediately tries to merge those new changes into your current code.

8. What does git merge do, and when do you use it?

A: git merge is the process of combining the history and changes from one branch into another branch. You typically use it when a feature developed on a separate branch is finished, reviewed, and needs to be integrated back into the main (e.g., main or master) branch.

9. How do you switch to another task without losing your current changes?

A: You use the git stash command. This command temporarily saves your uncommitted changes onto a stack, cleaning your working directory. You can then switch branches to work on the new task. When you return, you use git stash pop to restore your changes.

10. What is a merge conflict, and when does it happen?

A: A merge conflict happens when Git cannot automatically combine changes during a merge. This occurs when two different developers modify the exact same lines of code in the same file. Git pauses the merge and requires a human developer to manually resolve the disagreement.

11. How do you resolve a merge conflict in a file like abc.java?

A:

1. You manually open the file (abc.java) and find the conflict markers (<<<<<<<, =======, >>>>>>>).
2. You edit the file to keep only the correct final code, removing the markers.
3. You use git add abc.java to mark the conflict as resolved.
4. Finally, you run git commit to finish the merge operation.

12. How do you undo your last commit without losing the changes?

A: You use the command git reset --soft HEAD~1. This command removes the last commit from the branch history but keeps all the file changes in your staging area. This allows you to immediately create a new, fixed commit without losing any work.

13. How do you revert a specific commit that caused issues in production?

A: You use the command git revert <commit_hash>. This is the safest method because it creates a new commit that completely undoes the changes of the problematic commit. This preserves the project history and is the preferred way to undo changes that have already been pushed to a shared remote repository.